



Department of Information and Communication Technology

Faculty of Technology

University of Ruhuna

Database Management Systems Practicum

ICT 1222

Assignment 01 – Mini Project

Group 14

Submitted to: Ms.Sandaruwani Pathirage

Submitted by: TG/2024/2095 - Hasaranga KHM
TG/2024/2096 - Himansana
TG/2024/2097 - Nirmal
TG/2024/2121 - Samindi

1. Introduction to the Problem

Faculty of Technology (FOT), University of Ruhuna handles hundreds of students, many departments, and many courses each semester. Before implementing this application there was not an efficient way of handling the academic management process and resulted in many important problems.

- Attendance management process was done separately for each lecture/course without any method of obtaining attendance percentage for each student or alerting lecturers when a particular student falls under 80 percent attendance requirement for examinations.
- The process of calculating continuous assessment marks from quizzes, projects, mid-exam was done manually and prone to errors.
- No method existed for automatically calculating whether a student is eligible to appear for final exams based on CA marks and attendance.
- GPA calculation for final exams was a lengthy process that needed manual calculation of weighted average of marks in all courses taken by the particular student.
- No role-based access control and all types of users (Dean, Lecturer, Technical Officer, Students) could use the application with equal permissions.
- Students could apply for medical leave but without any systematic method of updating their attendance.
- For repeat students who take the same course twice, there was no system for management of this problem.

This is what led to the creation of the FOT Academic Management System, which is a completely normalized, relational database management system based on MySQL that solves all the above problems in an organized way..

2. Introduction to the Solution

The FOT Academic Management System database is a relational database management system implemented on MySQL database. The implementation has been done within four functional modules that are highly integrated:

- **Users Management** – controls users in the database through generalization hierarchy starting from PERSON, then specialized to STUDENT and STAFF, where STAFF itself gets specialized further into ADMIN, DEAN, LECTURER, and TECHNICAL_OFFICER. A person can have a USER_ACCOUNT associated with his or her record for logging into the system.
- **Academic Management** – includes DEPARTMENT, SEMESTER, COURSE_UNIT, COURSE_OFFERING (which is an offering of a particular course unit for a specific batch in a semester), COURSE_COMPONENT (Theory/Practical), and ENROLLMENT (M:N relationship between STUDENTS and COURSE_OFFERINGS).
- **Attendance Management** – consists of SESSION information for each component, ATTENDANCE_RECORD for each individual session (for example, present/absent/medical), and MEDICAL_RECORD for the medically approved absence.
- **Marks & Results Management** – holds ASSESSMENT_SCHEME (marks scheme for each course_offering), STUDENT_MARK (raw marks), ELIGIBILITY_RECORD (attendance percentage and CA marks), FINAL_RESULT (final mark, grade, and result code), GPA_RECORD (SGPA and CGPA), and GRADE_SCALE (mark to grade conversion table).

Referential integrity is enforced by foreign keys, data quality is maintained using constraints such as NOT NULL, UNIQUE, and ENUM, derived data is revealed using 15 database views, and all the business rules are automatically handled through three stored procedures. MySQL users have role-based accounts that adhere to the least privilege principle.

3. Proposed ER / EER Diagram

ER/EER diagrams were created using draw.io, and it captures all the entities, attributes, relationships, cardinalities, participation constraints, and specialization hierarchy of the FOT Academic Management system. Below are some of the decisions made during the designing of the diagram.

3.1 Generalisation / Specialisation Hierarchy

The supertype entity is PERSON and it includes the following attributes: Person_ID, F_Name, L_Name, NIC, Gender, DOB, Phone_no, and Address. The PERSON entity is then specialized into STUDENT and STAFF disjointly and partially. Additionally, STAFF is specialized disjointly and totally into ADMIN, DEAN, LECTURER, and TECHNICAL OFFICER entities.

3.2 Academic Core

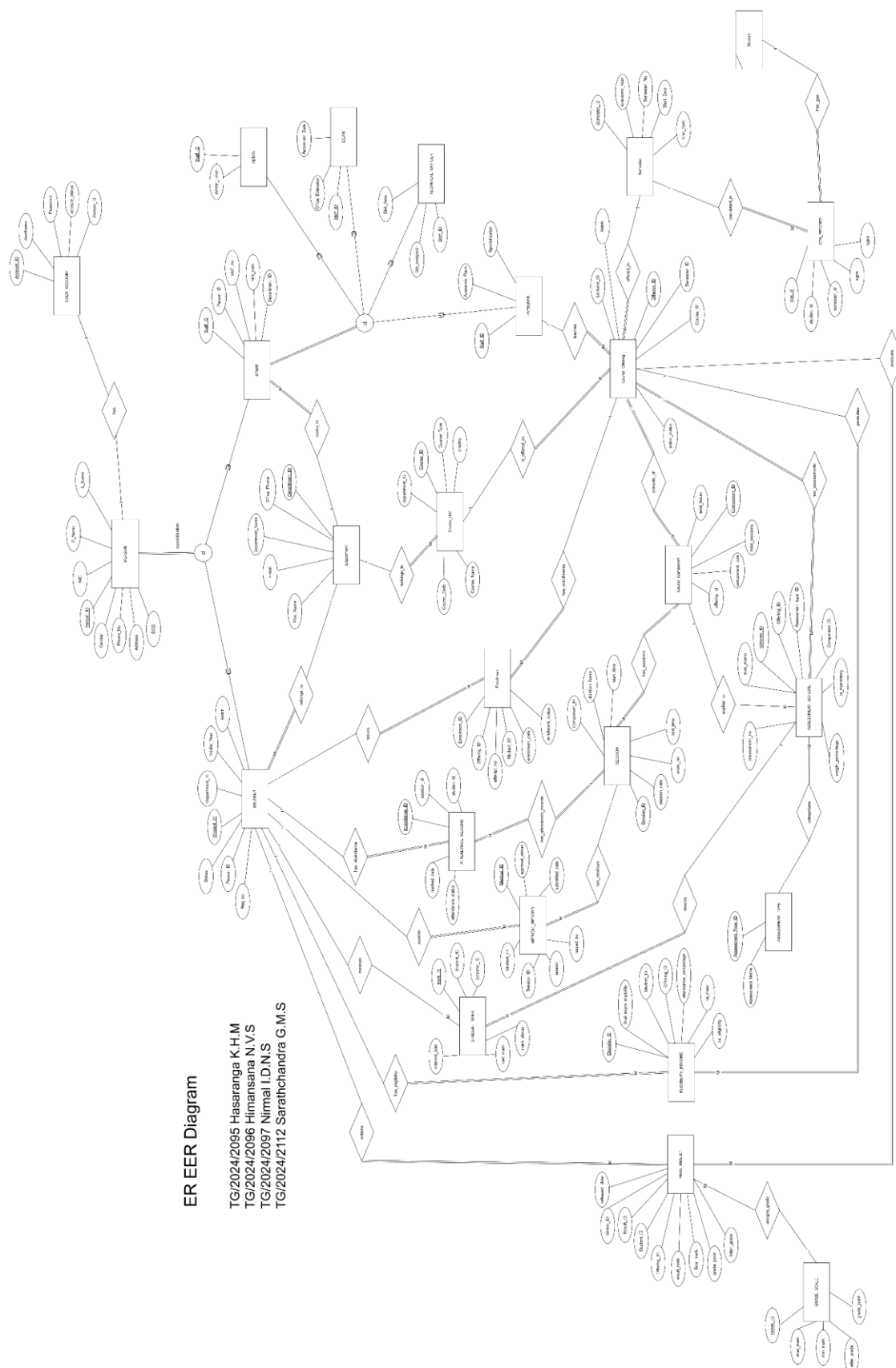
DEPARTMENT (Department_ID, Department_Name, Hod_Name, Email, Office_Phone) is a root entity. COURSE_UNIT is associated with a DEPARTMENT. A COURSE_OFFERING denotes a particular offering of a COURSE_UNIT during a particular SEMESTER, by a LECTURER, for a particular Batch. Each COURSE_OFFERING consists of one or more COURSE_COMPONENTs of type Theory/Practical. The ENROLLMENT entity handles the M:N relationship between STUDENT and COURSE_OFFERING along with enrollment_date, attempt_no, and enrollment_status.

3.3 Attendance Management

Each COURSE_COMPONENT has multiple SESSIONs (Session_ID, session_date, week_no, start_time, end_time, duration_hours). Per session, each enrolled STUDENT has an ATTENDANCE_RECORD (attendance_status: Present / Absent / Medical, marked_by linking to STAFF). Students may submit MEDICAL_RECORDs linked to specific absent sessions, with an approval_status field.

3.4 Marks & Results

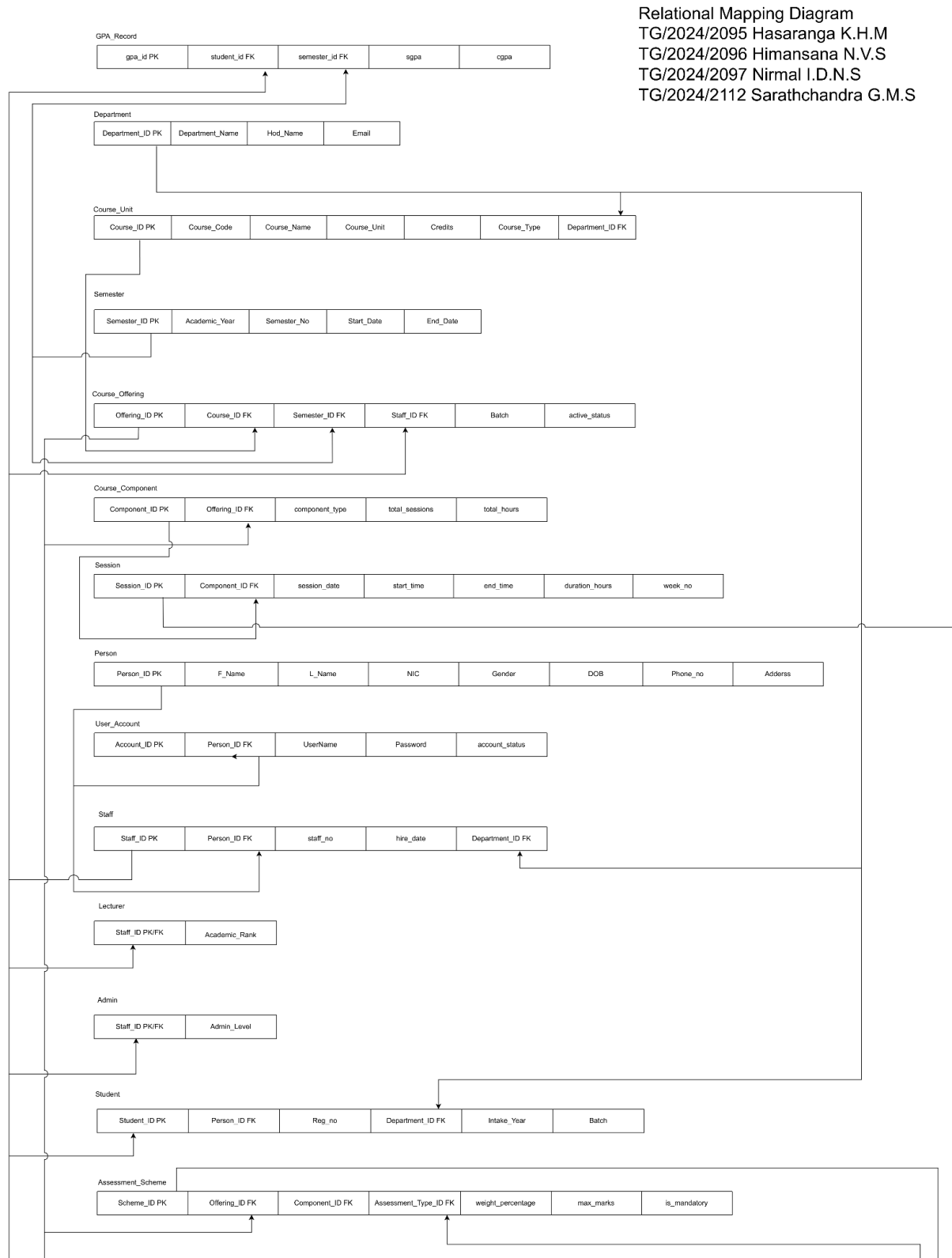
An ASSESSMENT_SCHEME defines the marking structure for each COURSE_OFFERING, linking to ASSESSMENT_TYPE (Quiz, Project, Mid Theory, Mid Practical, Final Theory, Final Practical) and optionally to a COURSE_COMPONENT. STUDENT_MARK stores the raw_mark entered by a LECTURER per student per scheme. ELIGIBILITY_RECORD is computed: attendance_percentage and ca_mark are derived, and eligibility flags are set. FINAL_RESULT stores the weighted final_mark and the letter_grade/grade_point assigned via GRADE_SCALE. GPA_RECORD holds the credit-weighted SGPA and CGPA per student per semester.

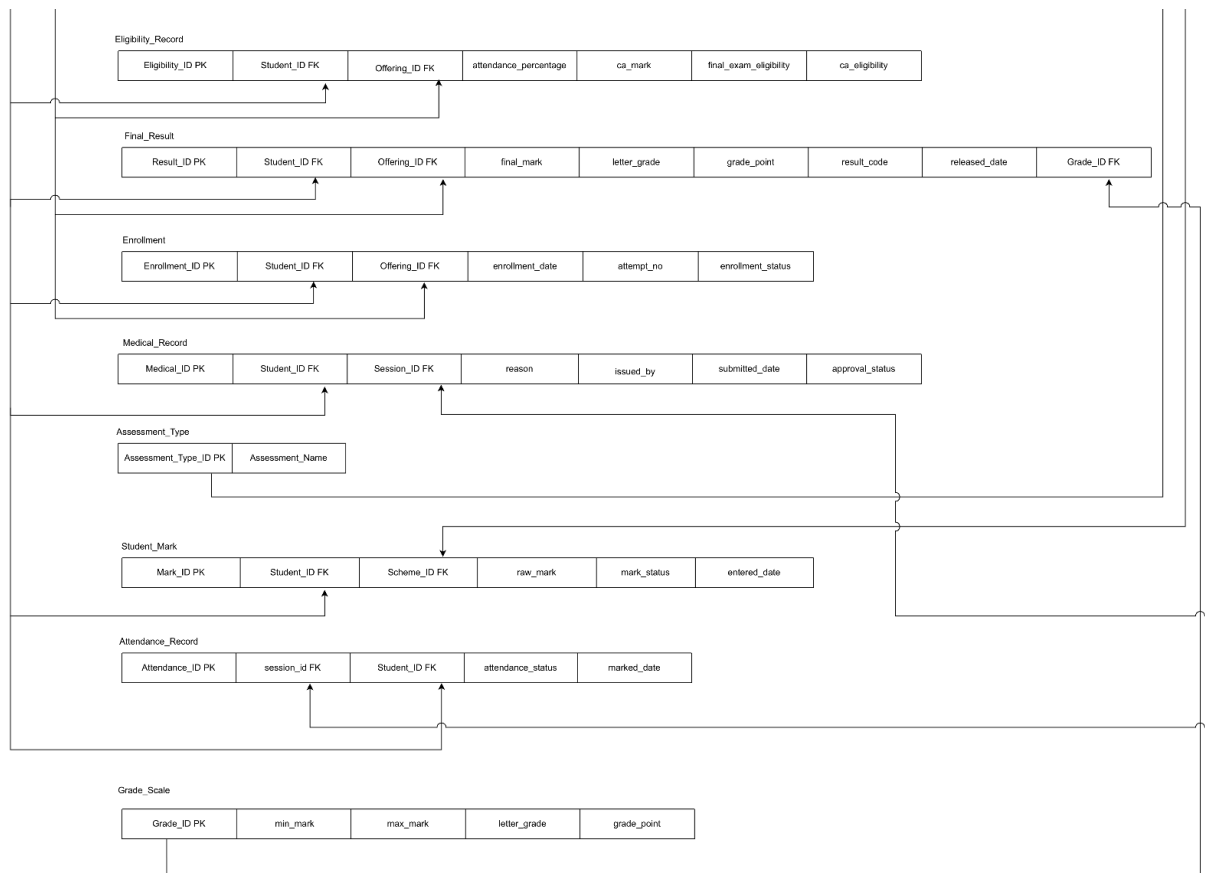


(Refer to the attached ER/EER diagram PDF for the complete visual diagram.) [link_ER_EER](#)

4. Proposed Relational Mapping Diagram

The relational mapping translates each entity and relationship from the ER/EER diagram into normalised relational tables (up to 3NF). The table below lists all 24 tables with their primary keys and key foreign key references.





(Refer to the attached relational mapping PDF for the complete visual diagram.) [link relational Mapping](#)

5. Table Structure of the Solution

This section presents the full DDL (CREATE TABLE) script ordered by dependency, followed by representative sample data inserts, and all database views with their actual SQL code.

5.1 Full Table Creation Script (Dependency-Ordered)

The following script creates all 24 tables in the correct order so that all foreign key references are satisfied. Each table is annotated with the team member who created it.

```
-- Created by: Nirmal (TG2097)
CREATE TABLE PERSON (
  Person_ID   VARCHAR(20) PRIMARY KEY,
  F_Name      VARCHAR(50) NOT NULL,
  L_Name      VARCHAR(50) NOT NULL,
  NIC         VARCHAR(20) NOT NULL UNIQUE,
  Gender      ENUM('Male', 'Female', 'Other') NOT NULL,
  DOB         DATE      NOT NULL,
  Phone_No    VARCHAR(15),
  Address     VARCHAR(255)
);
```

Person_ID	F_Name	L_Name	NIC	Gender	DOB	Phone_No	Address
P01	Nalin	Bandara	198011111V	Male	1980-01-10	0771000001	Matara
P02	Saman	Kumara	197522222V	Male	1975-05-20	0772000001	Galle
P03	Amali	Perera	198533331V	Female	1985-02-15	0773000001	Colombo
P04	Kasun	Kalhara	198633332V	Male	1986-03-15	0773000002	Matara
P05	Ruwan	Pathirana	198733333V	Male	1987-04-15	0773000003	Galle
P06	Nuwan	Pradeep	198833334V	Male	1988-05-15	0773000004	Tangalle
P07	Sanduni	Silva	198933335V	Female	1989-06-15	0773000005	Colombo
P08	Gayana	Madushanka	199033336V	Male	1990-07-15	0773000006	Matara
P09	Chathurika	Peiris	199133337V	Female	1991-08-15	0773000007	Galle
P10	Nilantha	Jayasooriya	199233338V	Male	1992-09-15	0773000008	Kandy

40 rows in set (0.30 sec)

```
-- Created by: Samindi (TG2112)
CREATE TABLE DEPARTMENT(
  Department_ID VARCHAR(20) PRIMARY KEY NOT NULL ,
  Department_Name VARCHAR(100) NOT NULL,
  Office_Phone INT(11),
  Email VARCHAR(100),
  Hod_Name VARCHAR(100)
);
```

Department_ID	Department_Name	Office_Phone	Email	Hod_Name
D01	Information and Communication Technology	412223334	hod_ict@fot.ruh.ac.lk	Dr. Dinithi
D02	Engineering Technology	412223335	hod_et@fot.ruh.ac.lk	Dr. Kamal Silva
D03	Biosystems Technology	412223336	hod_bst@fot.ruh.ac.lk	Dr. Nimal Fernando
D04	Multidisciplinary Studies	412223337	hod_mds@fot.ruh.ac.lk	Prof. Subash

4 rows in set (0.02 sec)

```
-- Created by: Samindi (TG2112)
CREATE TABLE SEMESTER(
  Semester_ID VARCHAR(20) PRIMARY KEY NOT NULL ,
  Academic_Year YEAR,
  Semester_No INT(5),
  Start_Date DATE,
  End_Date DATE
);
```

```
+-----+-----+-----+-----+-----+
| Semester_ID | Academic_Year | Semester_No | Start_Date | End_Date |
+-----+-----+-----+-----+-----+
| SEM01       | 2024          | 1           | 2026-05-04 | 2026-08-20 |
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

```
-- Created by: Hasaranga (TG2095)
CREATE TABLE ASSESSMENT TYPE (
  Assessment_Type_ID VARCHAR(20) PRIMARY KEY,
  Assessment_Name VARCHAR(50)
);
```

```
+-----+-----+
| Assessment_Type_ID | Assessment_Name |
+-----+-----+
| AT_FP              | Final Practical |
| AT_FT              | Final Theory    |
| AT_MP              | Mid Practical   |
| AT_MT              | Mid Theory      |
| AT_PR              | Project         |
| AT_QZ              | Quiz           |
+-----+-----+
6 rows in set (0.02 sec)
```

```
-- Created by: Hasaranga (TG2095)
CREATE TABLE GRADE_SCALE (
  Grade_ID VARCHAR(20) PRIMARY KEY,
  min_mark INT,
  max_mark INT,
  grade_point DECIMAL(3,2),
  letter_grade VARCHAR(5)
);
```

```
+-----+-----+-----+-----+-----+
| Grade_ID | min_mark | max_mark | grade_point | letter_grade |
+-----+-----+-----+-----+-----+
| G_A      | 75       | 84       | 4.00       | A            |
| G_AM     | 70       | 74       | 3.70       | A-           |
| G_AP     | 85       | 100      | 4.00       | A+           |
| G_B      | 60       | 64       | 3.00       | B            |
| G_BM     | 55       | 59       | 2.70       | B-           |
| G_BP     | 65       | 69       | 3.30       | B+           |
| G_C      | 45       | 49       | 2.00       | C            |
| G_CM     | 40       | 44       | 1.70       | C-           |
| G_CP     | 50       | 54       | 2.30       | C+           |
| G_D      | 35       | 39       | 1.30       | D            |
| G_E      | 0        | 34       | 0.00       | E            |
+-----+-----+-----+-----+-----+
11 rows in set (0.01 sec)
```



```
-- Created by: Nirmal (TG2097)
```

```
CREATE TABLE STAFF (
  Staff_ID    VARCHAR(20) NOT NULL,
  Person_ID   VARCHAR(20) NOT NULL,
  staff_no    VARCHAR(20) NOT NULL UNIQUE,
  hire_date   DATE      NOT NULL,
  Department_ID VARCHAR(20) NOT NULL,
  PRIMARY KEY (Staff_ID),
  FOREIGN KEY (Person_ID) REFERENCES PERSON(Person_ID),
  FOREIGN KEY (Department_ID) REFERENCES DEPARTMENT(Department_ID)
);
```

```
+-----+-----+-----+-----+-----+
| Staff_ID | Person_ID | staff_no | hire_date | Department_ID |
+-----+-----+-----+-----+-----+
| ST01     | P01       | EMP001   | 2010-01-01 | D01           |
| ST02     | P02       | EMP002   | 2005-01-01 | D01           |
| ST03     | P03       | EMP003   | 2015-01-01 | D01           |
| ST04     | P04       | EMP004   | 2016-01-01 | D01           |
| ST05     | P05       | EMP005   | 2017-01-01 | D02           |
| ST06     | P06       | EMP006   | 2018-01-01 | D03           |
| ST07     | P07       | EMP007   | 2019-01-01 | D01           |
| ST08     | P08       | EMP008   | 2019-06-01 | D02           |
| ST09     | P09       | EMP009   | 2020-01-01 | D03           |
| ST10     | P10       | EMP010   | 2020-06-01 | D01           |
| ST11     | P11       | EMP011   | 2020-01-01 | D01           |
| ST12     | P12       | EMP012   | 2020-02-01 | D01           |
| ST13     | P13       | EMP013   | 2021-01-01 | D02           |
| ST14     | P14       | EMP014   | 2021-05-01 | D03           |
| ST15     | P15       | EMP015   | 2022-01-01 | D01           |
+-----+-----+-----+-----+-----+
15 rows in set (0.003 sec)
```

```
-- Created by: Nirmal (TG2097)
```

```
CREATE TABLE STUDENT (
  Student_ID  VARCHAR(20) NOT NULL,
  Person_ID   VARCHAR(20) NOT NULL,
  Reg_no      VARCHAR(20) NOT NULL UNIQUE,
  Department_ID VARCHAR(20) NOT NULL,
  Intake_Year INT      NOT NULL,
  Batch       VARCHAR(20) NOT NULL,
  Status      VARCHAR(20),
  PRIMARY KEY (Student_ID),
  FOREIGN KEY (Person_ID) REFERENCES PERSON(Person_ID),
  FOREIGN KEY (Department_ID) REFERENCES DEPARTMENT(Department_ID)
);
```

```
+-----+-----+-----+-----+-----+-----+-----+
| Student_ID | Person_ID | Reg_no   | Department_ID | Intake_Year | Batch | Status |
+-----+-----+-----+-----+-----+-----+-----+
| S01        | P16       | TG/2024/2095 | D01           | 2024        | 23/24 | Proper |
| S02        | P17       | TG/2024/2096 | D01           | 2024        | 23/24 | Proper |
| S03        | P18       | TG/2024/2097 | D01           | 2024        | 23/24 | Proper |
| S04        | P19       | TG/2024/2112 | D01           | 2024        | 23/24 | Proper |
| S05        | P20       | TG/2024/2113 | D01           | 2024        | 23/24 | Proper |
| S06        | P21       | TG/2024/2114 | D01           | 2024        | 23/24 | Proper |
| S07        | P22       | TG/2024/2115 | D01           | 2024        | 23/24 | Proper |
| S08        | P23       | TG/2024/2116 | D01           | 2024        | 23/24 | Proper |
| S09        | P24       | TG/2024/2117 | D01           | 2024        | 23/24 | Proper |
+-----+-----+-----+-----+-----+-----+-----+
25 rows in set (0.001 sec)
```

```
-- Created by: Nirmal (TG2097)
CREATE TABLE USER_ACCOUNT (
  Account_ID VARCHAR(20) NOT NULL,
  Person_ID VARCHAR(20) NOT NULL,
  UserName VARCHAR(50) NOT NULL UNIQUE,
  Password VARCHAR(255) NOT NULL,
  account_status VARCHAR(20) NOT NULL,
  PRIMARY KEY (Account_ID),
  FOREIGN KEY (Person_ID) REFERENCES PERSON(Person_ID)
);
```

```
MariaDB [fot_academic_management_db]> SELECT * FROM USER_ACCOUNT;+-----+-----+-----+-----+-----+
----+
| Account_ID | Person_ID | UserName      | Password | account_status |
+-----+-----+-----+-----+-----+
| U01        | P01       | admin_nalin   | pass123  | Active         |
| U02        | P02       | dean_saman    | pass123  | Active         |
| U03        | P03       | lec_amali     | pass123  | Active         |
| U04        | P04       | lec_kasun     | pass123  | Active         |
| U05        | P05       | lec_ruwan     | pass123  | Active         |
| U06        | P06       | lec_nuwan     | pass123  | Active         |
| U07        | P07       | lec_sanduni   | pass123  | Active         |
| U08        | P08       | lec_gayan     | pass123  | Active         |
| U09        | P09       | lec_chathu    | pass123  | Active         |
| U10        | P10       | lec_nilantha  | pass123  | Active         |
+-----+-----+-----+-----+-----+
40 rows in set (0.002 sec)
```

```
-- Created by: Samindi (TG2112)
CREATE TABLE COURSE_UNIT(
  Course_ID VARCHAR(20) PRIMARY KEY NOT NULL,
  Course Code VARCHAR(10) NOT NULL,
  Course_Name VARCHAR(50) NOT NULL,
  Credits INT,
  Course Type VARCHAR(20),
  Department ID VARCHAR(20),
  FOREIGN KEY (Department_ID) REFERENCES DEPARTMENT(Department_ID)
);
```

```
+-----+-----+-----+-----+-----+
| Course_ID | Course_Code | Course_Name      | Credits | Course_Type | Department_ID |
+-----+-----+-----+-----+-----+
| CU_ENG1212 | ENG1212     | English II       | 2       | Theory      | D01           |
| CU_ICT1212 | ICT1212     | Database Management Systems | 2       | Theory      | D01           |
| CU_ICT1222 | ICT1222     | DBMS Practicum  | 2       | Practical   | D01           |
| CU_ICT1232 | ICT1232     | Web Development  | 2       | Theory      | D01           |
| CU_ICT1242 | ICT1242     | Web Development Practicum | 2       | Practical   | D01           |
| CU_ICT1252 | ICT1252     | OS Concepts and Application | 2       | Theory      | D01           |
| CU_ICT1261 | ICT1261     | System Prog. Fundamentals & Linux | 2       | Both        | D01           |
| CU_TCS1212 | TCS1212     | Fundamentals of Management | 2       | Theory      | D01           |
| CU_TMS1233 | TMS1233     | Discrete Mathematics | 3       | Theory      | D01           |
+-----+-----+-----+-----+-----+
9 rows in set (0.001 sec)
```

```
-- Created by: Nirmal (TG2097)
CREATE TABLE ADMIN (
  Staff_ID  VARCHAR(20) NOT NULL,
  Admin_Level VARCHAR(50) NOT NULL,
  PRIMARY KEY (Staff_ID),
  FOREIGN KEY (Staff_ID) REFERENCES STAFF(Staff_ID)
);
```

```
+-----+-----+
| Staff_ID | Admin_Level |
+-----+-----+
| ST01     | Super Admin |
+-----+-----+
1 row in set (0.004 sec)
```

```
-- Created by: Nirmal (TG2097)
CREATE TABLE DEAN (
  Staff_ID  VARCHAR(20) NOT NULL,
  Appointed_Date DATE NOT NULL,
  PRIMARY KEY (Staff_ID),
  FOREIGN KEY (Staff_ID) REFERENCES STAFF(Staff_ID)
);
```

```
+-----+-----+
| Staff_ID | Appointed_Date |
+-----+-----+
| ST02     | 2024-01-01     |
+-----+-----+
1 row in set (0.003 sec)
```

```
-- Created by: Nirmal (TG2097)
CREATE TABLE LECTURER (
  Staff_ID  VARCHAR(20) NOT NULL,
  Academic_Rank VARCHAR(50) NOT NULL,
  PRIMARY KEY (Staff_ID),
  FOREIGN KEY (Staff_ID) REFERENCES STAFF(Staff_ID)
);
```

```
+-----+-----+
| Staff_ID | Academic_Rank |
+-----+-----+
| ST03     | Senior Lecturer |
| ST04     | Lecturer        |
| ST05     | Senior Lecturer |
| ST06     | Lecturer        |
| ST07     | Lecturer        |
| ST08     | Senior Lecturer |
| ST09     | Lecturer        |
| ST10     | Lecturer        |
+-----+-----+
8 rows in set (0.003 sec)
```

```
-- Created by: Nirmal (TG2097)
CREATE TABLE TECHNICAL_OFFICER (
    Staff_ID          VARCHAR(20) NOT NULL,
    Specialization    VARCHAR(100) NOT NULL,
    lab_assigned      VARCHAR(100),
    PRIMARY KEY (Staff_ID),
    FOREIGN KEY (Staff_ID) REFERENCES STAFF(Staff_ID)
);
```

```
+-----+-----+-----+
| Staff_ID | Specialization | lab_assigned |
+-----+-----+-----+
| ST11     | Networking     | Lab A       |
| ST12     | Hardware       | Lab B       |
| ST13     | Software       | Lab C       |
| ST14     | Hardware       | Lab D       |
| ST15     | Networking     | Lab E       |
+-----+-----+-----+
5 rows in set (0.004 sec)
```

```
-- Created by: Hasaranga (TG2095)
CREATE TABLE COURSE_OFFERING (
    Offering_ID VARCHAR(20) PRIMARY KEY NOT NULL,
    Course_ID VARCHAR(20) NOT NULL,
    Lecturer_ID VARCHAR(20) NOT NULL,
    Semester_ID VARCHAR(20) NOT NULL,
    Batch VARCHAR(10),
    active_status BOOLEAN,
    FOREIGN KEY (Course_ID) REFERENCES COURSE_UNIT(Course_ID),
    FOREIGN KEY (Lecturer_ID) REFERENCES LECTURER(Staff_ID),
    FOREIGN KEY (Semester_ID) REFERENCES SEMESTER(Semester_ID)
);
```

```
+-----+-----+-----+-----+-----+-----+
| Offering_ID | Course_ID | Lecturer_ID | Semester_ID | Batch | active_status |
+-----+-----+-----+-----+-----+-----+
| OFF_DBP    | CU_ICT1222 | ST05        | SEM01       | 2022/2023 | 1 |
| OFF_DBS    | CU_ICT1212 | ST10        | SEM01       | 2022/2023 | 1 |
| OFF_ENG    | CU_ENG1212 | ST09        | SEM01       | 2022/2023 | 1 |
| OFF_LIN    | CU_ICT1261 | ST04        | SEM01       | 2022/2023 | 1 |
| OFF_MAT    | CU_TMS1233 | ST08        | SEM01       | 2022/2023 | 1 |
| OFF_MGT    | CU_TCS1212 | ST03        | SEM01       | 2022/2023 | 1 |
| OFF_OSC    | CU_ICT1252 | ST09        | SEM01       | 2022/2023 | 1 |
| OFF_WEB    | CU_ICT1232 | ST06        | SEM01       | 2022/2023 | 1 |
| OFF_WEP    | CU_ICT1242 | ST07        | SEM01       | 2022/2023 | 1 |
+-----+-----+-----+-----+-----+-----+
9 rows in set (0.000 sec)
```

```
-- Created by: Hasaranga (TG2095)
CREATE TABLE COURSE_COMPONENT (
  Component_ID VARCHAR(20) PRIMARY KEY NOT NULL,
  offering_id VARCHAR(20) NOT NULL,
  component_type VARCHAR(20),
  total_sessions INT,
  total_hours INT,
  FOREIGN KEY (offering_id) REFERENCES COURSE_OFFERING(Offering_ID)
);
```

Component_ID	offering_id	component_type	total_sessions	total_hours
CC_DBP	OFF_DBP	Practical	15	30
CC_DBS	OFF_DBS	Theory	15	30
CC_ENG	OFF_ENG	Theory	15	30
CC_LIN_P	OFF_LIN	Practical	15	15
CC_LIN_T	OFF_LIN	Theory	15	15
CC_MAT	OFF_MAT	Theory	15	30
CC_MGT	OFF_MGT	Theory	15	30
CC_OSC	OFF_OSC	Theory	15	30
CC_WEB_P1	OFF_WEP	Practical	15	30
CC_WEB_P2	OFF_WEP	Practical	15	45
CC_WEB_T	OFF_WEB	Theory	15	15

11 rows in set (0.001 sec)

```
-- Created by: Hasaranga (TG2095)
CREATE TABLE ENROLLMENT (
  Enrollment_ID VARCHAR(20) PRIMARY KEY NOT NULL,
  Student_ID VARCHAR(20) NOT NULL,
  Offering_ID VARCHAR(20) NOT NULL,
  enrollment_date DATE,
  attempt_no INT,
  enrollment_status VARCHAR(20),
  FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
  FOREIGN KEY (Offering_ID) REFERENCES COURSE_OFFERING(Offering_ID)
);
```

Enrollment_ID	Student_ID	Offering_ID	enrollment_date	attempt_no	enrollment_status
E_DBP_01	S01	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_02	S02	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_03	S03	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_04	S04	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_05	S05	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_06	S06	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_07	S07	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_08	S08	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_09	S09	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_10	S10	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_11	S11	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_12	S12	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_13	S13	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_14	S14	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_15	S15	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_16	S16	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_17	S17	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_18	S18	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_19	S19	OFF_DBP	2026-05-01	1	Enrolled
E_DBP_20	S20	OFF_DBP	2026-05-01	1	Enrolled

225 rows in set (0.004 sec)

```
-- Created by: Samindi (TG2112)
```

```
CREATE TABLE SESSION (
  Session_ID VARCHAR(20) PRIMARY KEY NOT NULL,
  Component ID VARCHAR(20) NOT NULL,
  session_date DATE,
  week_no INT,
  start_time TIME,
  end_time TIME,
  duration_hours INT,
  FOREIGN KEY (Component ID) REFERENCES COURSE_COMPONENT(Component ID)
);
```

Session_ID	Component_ID	session_date	week_no	start_time	end_time	duration_hours
W01_DBP	CC_DBP	2026-05-05	1	09:00:00	12:00:00	3
W01_DBS	CC_DBS	2026-05-07	1	10:00:00	12:00:00	2
W01_ENG	CC_ENG	2026-05-07	1	09:00:00	10:00:00	1
W01_LIN	CC_LIN_T	2026-05-04	1	13:00:00	15:00:00	2
W01_MAT	CC_MAT	2026-05-06	1	09:00:00	12:00:00	3
W01_MGT	CC_MGT	2026-05-04	1	09:00:00	11:00:00	2
W01_OSC	CC_OSC	2026-05-08	1	08:00:00	10:00:00	2
W01_WEBP1	CC_WEB_P1	2026-05-05	1	14:00:00	16:00:00	2
W01_WEBP2	CC_WEB_P2	2026-05-07	1	13:00:00	16:00:00	3
W01_WEBT	CC_WEB_T	2026-05-05	1	13:00:00	14:00:00	1
W02_DBP	CC_DBP	2026-05-12	2	09:00:00	12:00:00	3
W02_DBS	CC_DBS	2026-05-14	2	10:00:00	12:00:00	2
W02_ENG	CC_ENG	2026-05-14	2	09:00:00	10:00:00	1
W02_LIN	CC_LIN_T	2026-05-11	2	13:00:00	15:00:00	2
W02_MAT	CC_MAT	2026-05-13	2	09:00:00	12:00:00	3
W02_MGT	CC_MGT	2026-05-11	2	09:00:00	11:00:00	2

150 rows in set (0.004 sec)

```
-- Created by: Hasaranga (TG2095)
```

```
CREATE TABLE GPA_RECORD (
  gpa_id VARCHAR(20) PRIMARY KEY NOT NULL,
  student_id VARCHAR(20) NOT NULL,
  semester_id VARCHAR(20) NOT NULL,
  sgpa DECIMAL(4,2),
  cgpa DECIMAL(4,2),
  FOREIGN KEY (student_id) REFERENCES STUDENT(Student_ID),
  FOREIGN KEY (semester_id) REFERENCES SEMESTER(Semester_ID)
);
```

gpa_id	student_id	semester_id	sgpa	cgpa
GPA_SEM01_S01	S01	SEM01	4.00	4.00
GPA_SEM01_S02	S02	SEM01	4.00	4.00
GPA_SEM01_S03	S03	SEM01	3.76	3.76
GPA_SEM01_S04	S04	SEM01	4.00	4.00
GPA_SEM01_S05	S05	SEM01	4.00	4.00
GPA_SEM01_S06	S06	SEM01	4.00	4.00
GPA_SEM01_S07	S07	SEM01	4.00	4.00
GPA_SEM01_S08	S08	SEM01	4.00	4.00
GPA_SEM01_S09	S09	SEM01	0.00	0.00
GPA_SEM01_S10	S10	SEM01	4.00	4.00
GPA_SEM01_S11	S11	SEM01	3.91	3.91
GPA_SEM01_S12	S12	SEM01	4.00	4.00
GPA_SEM01_S13	S13	SEM01	0.00	0.00

25 rows in set (0.001 sec)

```
-- Created by: Hasaranga (TG2095)
```

```
CREATE TABLE ASSESSMENT SCHEME (
  Scheme_ID VARCHAR(20) PRIMARY KEY NOT NULL,
  Offering_ID VARCHAR(20) NOT NULL,
  Assessment_Type_ID VARCHAR(20) NOT NULL,
  Component_ID VARCHAR(20),
  weight_percentage DECIMAL(5,2),
  assessment_no INT,
  max_marks INT,
  is_mandatory BOOLEAN,
  FOREIGN KEY (Offering_ID) REFERENCES COURSE_OFFERING(Offering_ID),
  FOREIGN KEY (Assessment_Type_ID) REFERENCES ASSESSMENT_TYPE(Assessment_Type_ID),
  FOREIGN KEY (Component_ID) REFERENCES COURSE_COMPONENT(Component_ID)
);
```

Scheme_ID	Offering_ID	Assessment_Type_ID	Component_ID	weight_percentage	assessment_no	max_marks	is_mandatory
SC_DBP_FP	OFF_DBP	AT_FP	CC_DBP	60.00	1	100	1
SC_DBP_MP	OFF_DBP	AT_MP	CC_DBP	20.00	1	100	1
SC_DBP_MT	OFF_DBP	AT_MT	CC_DBP	20.00	1	100	1
SC_DBP_PR	OFF_DBP	AT_PR	CC_DBP	10.00	1	100	1
SC_DBP_QZ	OFF_DBP	AT_QZ	CC_DBP	10.00	1	100	1
SC_DBS_FT	OFF_DBS	AT_FT	CC_DBS	70.00	1	100	1
SC_DBS_MT	OFF_DBS	AT_MT	CC_DBS	15.00	1	100	1
SC_DBS_PR	OFF_DBS	AT_PR	CC_DBS	10.00	1	100	1
SC_DBS_QZ	OFF_DBS	AT_QZ	CC_DBS	5.00	1	100	1
SC_ENG_FT	OFF_ENG	AT_FT	CC_ENG	70.00	1	100	1
SC_ENG_MT	OFF_ENG	AT_MT	CC_ENG	15.00	1	100	1
SC_ENG_PR	OFF_ENG	AT_PR	CC_ENG	10.00	1	100	1

38 rows in set (0.001 sec)

```
-- Created by: Samindi (TG2112)
```

```
CREATE TABLE ATTENDANCE RECORD (
  Attendance_ID VARCHAR(20) PRIMARY KEY NOT NULL,
  session_id VARCHAR(20) NOT NULL,
  student_id VARCHAR(20) NOT NULL,
  marked_by VARCHAR(20) NOT NULL,
  attendance_status VARCHAR(20),
  marked_date DATE,
  FOREIGN KEY (session_id) REFERENCES SESSION(Session_ID),
  FOREIGN KEY (student_id) REFERENCES STUDENT(Student_ID),
  FOREIGN KEY (marked_by) REFERENCES STAFF(Staff_ID)
);
```

Attendance_ID	session_id	student_id	marked_by	attendance_status	marked_date
A_W01_DBP_01	W01_DBP	S01	ST05	Present	2026-05-05
A_W01_DBP_02	W01_DBP	S02	ST05	Present	2026-05-05
A_W01_DBP_03	W01_DBP	S03	ST05	Present	2026-05-05
A_W01_DBP_04	W01_DBP	S04	ST05	Present	2026-05-05
A_W01_DBP_05	W01_DBP	S05	ST05	Present	2026-05-05
A_W01_DBP_06	W01_DBP	S06	ST05	Present	2026-05-05
A_W01_DBP_07	W01_DBP	S07	ST05	Present	2026-05-05
A_W01_DBP_08	W01_DBP	S08	ST05	Present	2026-05-05
A_W01_DBP_09	W01_DBP	S09	ST05	Present	2026-05-05
A_W01_DBP_10	W01_DBP	S10	ST05	Present	2026-05-05
A_W01_DBS_01	W01_DBS	S01	ST10	Present	2026-05-07

```

+-----+-----+-----+-----+-----+-----+
| A_W01_DBS_02 | W01_DBS | S02 | ST10 | Present | 2026-05-07 |
| A_W01_DBS_03 | W01_DBS | S03 | ST10 | Present | 2026-05-07 |
| A_W01_DBS_04 | W01_DBS | S04 | ST10 | Medical | 2026-05-07 |
| A_W01_DBS_05 | W01_DBS | S05 | ST10 | Present | 2026-05-07 |
| A_W01_DBS_06 | W01_DBS | S06 | ST10 | Present | 2026-05-07 |
| A_W01_DBS_07 | W01_DBS | S07 | ST10 | Present | 2026-05-07 |
+-----+-----+-----+-----+-----+
3000 rows in set (0.021 sec)

```

```
-- Created by: Samindi (TG2112)
```

```

CREATE TABLE MEDICAL_RECORD (
  Medical_ID VARCHAR(20) PRIMARY KEY NOT NULL,
  Student_ID VARCHAR(20) NOT NULL,
  Session_ID VARCHAR(20) NOT NULL,
  reason VARCHAR(255),
  issued_by VARCHAR(100),
  submitted_date DATE,
  approval_status VARCHAR(20),
  FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
  FOREIGN KEY (Session_ID) REFERENCES SESSION(Session_ID)
);

```

```

+-----+-----+-----+-----+-----+-----+-----+
| Medical_ID | Student_ID | Session_ID | reason | issued_by | submitted_date | approval_status |
+-----+-----+-----+-----+-----+-----+-----+
| MED_001 | S04 | W01_DBS | Viral Fever | General Hospital Matara | 2026-05-10 | Approved |
| MED_002 | S04 | W02_LIN | Dengue Fever | General Hospital Matara | 2026-05-15 | Approved |
| MED_003 | S07 | W03_MAT | Severe Migraine | University Medical Center | 2026-05-25 | Approved |
| MED_004 | S09 | W06_LIN | Food Poisoning | Dr. Perera (Private Clinic) | 2026-06-10 | Approved |
| MED_005 | S08 | W10_MGT | Sports Injury (Ankle) | Sports Medical Unit | 2026-07-08 | Pending |
+-----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.001 sec)

```

```
-- Created by: Hasaranga (TG2095)
```

```

CREATE TABLE STUDENT_MARK (
  Mark_ID VARCHAR(20) PRIMARY KEY NOT NULL,
  Student_ID VARCHAR(20) NOT NULL,
  Scheme_ID VARCHAR(20) NOT NULL,
  entered_by VARCHAR(20) NOT NULL,
  raw_mark DECIMAL(5,2),
  mark_status VARCHAR(20),
  entered_date DATE,
  FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
  FOREIGN KEY (Scheme_ID) REFERENCES ASSESSMENT_SCHEME(Scheme_ID),
  FOREIGN KEY (entered_by) REFERENCES STAFF(Staff_ID)
);

```

```

+-----+-----+-----+-----+-----+-----+-----+
| Mark_ID | Student_ID | Scheme_ID | entered_by | raw_mark | mark_status | entered_date |
+-----+-----+-----+-----+-----+-----+-----+
| M_DBPF_01 | S01 | SC_DBP_FP | ST05 | 90.00 | Verified | 2026-08-20 |
| M_DBPF_02 | S02 | SC_DBP_FP | ST05 | 86.00 | Verified | 2026-08-20 |
| M_DBPF_03 | S03 | SC_DBP_FP | ST05 | 80.00 | Verified | 2026-08-20 |
| M_DBPF_06 | S06 | SC_DBP_FP | ST05 | 82.00 | Verified | 2026-08-20 |
| M_DBPF_07 | S07 | SC_DBP_FP | ST05 | 90.00 | Verified | 2026-08-20 |
| M_DBPF_08 | S08 | SC_DBP_FP | ST05 | 94.00 | Verified | 2026-08-20 |
| M_DBPF_09 | S09 | SC_DBP_FP | ST05 | 0.00 | Verified | 2026-08-20 |
| M_DBPF_10 | S10 | SC_DBP_FP | ST05 | 85.00 | Verified | 2026-08-20 |
+-----+-----+-----+-----+-----+-----+-----+
900 rows in set (0.002 sec)

```



```
-- Created by: Hasaranga (TG2095)
CREATE TABLE ELIGIBILITY_RECORD (
  Eligibility_ID VARCHAR(20) PRIMARY KEY NOT NULL,
  Student_ID VARCHAR(20) NOT NULL,
  Offering_ID VARCHAR(20) NOT NULL,
  attendance_percentage DECIMAL(5,2),
  ca_mark DECIMAL(5,2),
  ca_eligibility BOOLEAN,
  final_exam_eligibility BOOLEAN,
  FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
  FOREIGN KEY (Offering_ID) REFERENCES COURSE_OFFERING(Offering_ID)
);
```

Eligibility_ID	Student_ID	Offering_ID	attendance_percentage	ca_mark	ca_eligibility	final_exam_eligibility
ELI_OFF_DBP_S01	S01	OFF_DBP	100.00	34.30	1	1
ELI_OFF_DBP_S02	S02	OFF_DBP	100.00	32.60	1	1
ELI_OFF_DBP_S03	S03	OFF_DBP	100.00	29.40	1	1
ELI_OFF_DBP_S04	S04	OFF_DBP	100.00	37.00	1	1
ELI_OFF_DBP_S05	S05	OFF_DBP	100.00	37.90	1	1
ELI_OFF_DBP_S06	S06	OFF_DBP	100.00	30.90	1	1
ELI_OFF_DBP_S07	S07	OFF_DBP	100.00	34.70	1	1
ELI_OFF_DBP_S08	S08	OFF_DBP	100.00	36.00	1	1
ELI_OFF_DBP_S09	S09	OFF_DBP	100.00	0.00	1	1
ELI_OFF_DBP_S10	S10	OFF_DBP	100.00	31.80	1	1

225 rows in set (0.001 sec)

```
-- Created by: Hasaranga (TG2095)
CREATE TABLE FINAL_RESULT (
  Result_ID VARCHAR(20) PRIMARY KEY NOT NULL,
  Student_ID VARCHAR(20) NOT NULL,
  Offering_ID VARCHAR(20) NOT NULL,
  final_mark DECIMAL(5,2),
  letter_grade VARCHAR(5),
  grade_point DECIMAL(3,2),
  result_code VARCHAR(10),
  released_date DATE,
  FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
  FOREIGN KEY (Offering_ID) REFERENCES COURSE_OFFERING(Offering_ID)
);
```

Result_ID	Student_ID	Offering_ID	final_mark	letter_grade	grade_point	result_code	released_date
RES_OFF_DBP_S01	S01	OFF_DBP	88.30	A+	4.00	PASS	2026-05-07
RES_OFF_DBP_S02	S02	OFF_DBP	84.20	A	4.00	PASS	2026-05-07
RES_OFF_DBP_S03	S03	OFF_DBP	77.40	A	4.00	PASS	2026-05-07
RES_OFF_DBP_S04	S04	OFF_DBP	94.00	A+	4.00	PASS	2026-05-07
RES_OFF_DBP_S05	S05	OFF_DBP	96.70	A+	4.00	PASS	2026-05-07
RES_OFF_DBP_S06	S06	OFF_DBP	80.10	A	4.00	PASS	2026-05-07
RES_OFF_DBP_S07	S07	OFF_DBP	88.70	A+	4.00	PASS	2026-05-07
RES_OFF_DBP_S08	S08	OFF_DBP	92.40	A+	4.00	PASS	2026-05-07
RES_OFF_DBP_S09	S09	OFF_DBP	0.00	E	0.00	FAIL	2026-05-07
RES_OFF_DBP_S10	S10	OFF_DBP	82.80	A	4.00	PASS	2026-05-07

225 rows in set (0.001 sec)

5.2 Database Views

The system defines 15 views across four categories. Students and Technical Officers access data exclusively through these views they have no direct access to base tables.

ATTENDANCE VIEWS created by Hasaranga (TG2095)

1. Batch_Attendance_Summary overall attendance % and eligibility per student per course.

```
CREATE VIEW Batch_Attendance_Summary AS
SELECT
    ST.Reg No,
    CU.Course_Code,
    CU.Course_Name,
    COUNT(S.Session_ID) AS Total_Sessions,
    SUM(CASE WHEN A.attendance_status IN ('Present', 'Medical') THEN 1 ELSE 0 END) AS Attended_Sessions,
    ROUND((SUM(CASE WHEN A.attendance_status IN ('Present', 'Medical') THEN 1 ELSE 0 END) /
COUNT(S.Session_ID)) * 100, 2) AS Attendance_Percentage,
    CASE
        WHEN (SUM(CASE WHEN A.attendance_status IN ('Present', 'Medical') THEN 1 ELSE 0 END) /
COUNT(S.Session_ID)) * 100 >= 80.00 THEN 'Eligible'
        ELSE 'Not Eligible'
    END AS Eligibility_Status
FROM STUDENT ST
JOIN ATTENDANCE_RECORD A ON ST.Student_ID = A.student_id
JOIN SESSION S ON A.session_id = S.Session_ID
JOIN COURSE_COMPONENT CC ON S.Component_ID = CC.Component_ID
JOIN COURSE_OFFERING CO ON CC.offering_id = CO.Offering_ID
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID
GROUP BY ST.Reg_No, CU.Course_Code, CU.Course_Name;
```

2. Component_Wise_Attendance attendance split by Theory / Practical component.

```
CREATE VIEW Component_Wise_Attendance AS
SELECT
    ST.Reg No,
    CU.Course_Code,
    CC.component_type,
    COUNT(S.Session_ID) AS Total_Sessions,
    SUM(CASE WHEN A.attendance_status IN ('Present', 'Medical') THEN 1 ELSE 0 END) AS Attended_Sessions,
    ROUND((SUM(CASE WHEN A.attendance_status IN ('Present', 'Medical') THEN 1 ELSE 0 END) /
COUNT(S.Session_ID)) * 100, 2) AS Component_Attendance_Percentage
FROM STUDENT ST
JOIN ATTENDANCE_RECORD A ON ST.Student_ID = A.student_id
JOIN SESSION S ON A.session_id = S.Session_ID
JOIN COURSE_COMPONENT CC ON S.Component_ID = CC.Component_ID
JOIN COURSE_OFFERING CO ON CC.offering_id = CO.Offering_ID
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID
GROUP BY ST.Reg_No, CU.Course_Code, CC.component_type;
```

3. Individual_Attendance_Summary attendance % per student per subject with name.

```
CREATE VIEW Individual_Attendance_Summary AS
SELECT
    ST.Reg_No,
    P.F_Name,
    P.L_Name,
    CU.Course_Code,
    ROUND((SUM(CASE WHEN A.attendance_status IN ('Present', 'Medical') THEN 1 ELSE 0 END) /
COUNT(S.Session_ID)) * 100, 2) AS Attendance_Percentage
FROM STUDENT ST
JOIN PERSON P ON ST.Person_ID = P.Person_ID
JOIN ATTENDANCE_RECORD A ON ST.Student_ID = A.student_id
JOIN SESSION S ON A.session_id = S.Session_ID
JOIN COURSE_COMPONENT CC ON S.Component_ID = CC.Component_ID
JOIN COURSE_OFFERING CO ON CC.offering_id = CO.Offering_ID
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID
GROUP BY ST.Reg_No, P.F_Name, P.L_Name, CU.Course_Code;
```

4. Individual_Subject_Attendance_Detail session-by-session attendance record per student.

```
CREATE VIEW Individual_Subject_Attendance_Detail AS
SELECT
    ST.Reg_No,
    CU.Course_Code,
    S.session_date,
    S.start_time,
    CC.component_type,
    A.attendance_status,
    A.marked_date
FROM STUDENT ST
JOIN ATTENDANCE_RECORD A ON ST.Student_ID = A.student_id
JOIN SESSION S ON A.session_id = S.Session_ID
JOIN COURSE_COMPONENT CC ON S.Component_ID = CC.Component_ID
JOIN COURSE_OFFERING CO ON CC.offering_id = CO.Offering_ID
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID;
```

MARKS VIEWS created by Samindi (TG2112)

5. Individual_CA_Marks_Detail each CA mark with weight, excluding final exams (AT_FT, AT_FP).

```
CREATE VIEW Individual_CA_Marks_Detail AS
SELECT
    ST.Reg_No,
    CU.Course_Code,
    ATY.Assessment_Name,
    SM.raw_mark,
    SCH.weight_percentage,
    ROUND((SM.raw_mark * (SCH.weight_percentage / 100)), 2) AS weighted_mark
FROM STUDENT_MARK SM
JOIN ASSESSMENT_SCHEME SCH ON SM.Scheme_ID = SCH.Scheme_ID
JOIN ASSESSMENT_TYPE ATY ON SCH.Assessment_Type_ID = ATY.Assessment_Type_ID
JOIN STUDENT ST ON SM.Student_ID = ST.Student_ID
JOIN COURSE_OFFERING CO ON SCH.Offering_ID = CO.Offering_ID
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID
WHERE ATY.Assessment_Type_ID NOT IN ('AT_FT', 'AT_FP');
```

6. Student_CA_Marks_Summary total CA mark and CA eligibility per student per course.

```
CREATE VIEW Student_CA_Marks_Summary AS
SELECT
    ST.Reg_No,
    P.F_Name,
    P.L_Name,
    CU.Course_Code,
    CU.Course_Name,
    ER.ca_mark,
    ER.ca_eligibility
FROM STUDENT ST
JOIN PERSON P ON ST.Person_ID = P.Person_ID
JOIN ELIGIBILITY_RECORD ER ON ST.Student_ID = ER.Student_ID
JOIN COURSE_OFFERING CO ON ER.Offering_ID = CO.Offering_ID
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID;
```

7. Batch_CA_Marks_Summary CA marks for the whole batch, ordered by course then registration number.

```
CREATE VIEW Batch_CA_Marks_Summary AS
SELECT
    CU.Course_Code,
    CU.Course_Name,
    ST.Reg_No,
    P.F_Name,
    P.L_Name,
    ER.ca_mark,
    ER.ca_eligibility
```

```

FROM ELIGIBILITY_RECORD ER
JOIN STUDENT ST ON ER.Student_ID = ST.Student_ID
JOIN PERSON P ON ST.Person_ID = P.Person_ID
JOIN COURSE_OFFERING CO ON ER.Offering_ID = CO.Offering_ID
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID
ORDER BY CU.Course_Code, ST.Reg_No;

```

8. Comprehensive_Marks_Breakdown all assessment marks pivoted to columns per student per course.

```

CREATE VIEW Comprehensive_Marks_Breakdown AS
SELECT
    ST.Reg_No,
    CU.Course_Code,
    CU.Course_Name,
    -- Quiz Marks
    MAX(CASE WHEN ATY.Assessment_Type_ID = 'AT_QZ' THEN SM.raw_mark ELSE NULL END)
AS Quiz_Mark,
    -- Project Marks
    MAX(CASE WHEN ATY.Assessment_Type_ID = 'AT_PR' THEN SM.raw_mark ELSE NULL END)
AS Project_Mark,
    -- Mid Theory Marks
    MAX(CASE WHEN ATY.Assessment_Type_ID = 'AT_MT' THEN SM.raw_mark ELSE NULL END)
AS Mid_Theory_Mark,
    -- Mid Practical Marks
    MAX(CASE WHEN ATY.Assessment_Type_ID = 'AT_MP' THEN SM.raw_mark ELSE NULL END)
AS Mid_Practical_Mark,
    -- Final Theory Marks
    MAX(CASE WHEN ATY.Assessment_Type_ID = 'AT_FT' THEN SM.raw_mark ELSE NULL END)
AS Final_Theory_Mark,
    -- Final Practical Marks
    MAX(CASE WHEN ATY.Assessment_Type_ID = 'AT_FP' THEN SM.raw_mark ELSE NULL END)
AS Final_Practical_Mark
FROM STUDENT ST
JOIN STUDENT_MARK SM ON ST.Student_ID = SM.Student_ID
JOIN ASSESSMENT_SCHEME SCH ON SM.Scheme_ID = SCH.Scheme_ID
JOIN ASSESSMENT_TYPE ATY ON SCH.Assessment_Type_ID = ATY.Assessment_Type_ID
JOIN COURSE_OFFERING CO ON SCH.Offering_ID = CO.Offering_ID
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID
GROUP BY ST.Reg_No, CU.Course_Code, CU.Course_Name;

```

9. Final_Marks_Summary attendance %, CA mark, final mark, grade, result code — all in one view.

```
CREATE VIEW Final_Marks_Summary AS
SELECT
    CU.Course_Code,
    CU.Course_Name,
    ST.Reg_No,
    P.F_Name,
    P.L_Name,
    ER.attendance_percentage,
    ER.ca_mark,
    FR.final_mark,
    FR.letter_grade,
    FR.grade_point,
    FR.result_code
FROM FINAL_RESULT FR
JOIN ELIGIBILITY_RECORD ER ON FR.Student_ID = ER.Student_ID AND FR.Offering_ID =
ER.Offering_ID
JOIN STUDENT ST ON FR.Student_ID = ST.Student_ID
JOIN PERSON P ON ST.Person_ID = P.Person_ID
JOIN COURSE_OFFERING CO ON FR.Offering_ID = CO.Offering_ID
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID;
```

ELIGIBILITY & GRADE VIEWS created by Himansana (TG2096)

10. Student_Eligibility_Summary attendance status, CA mark, and both eligibility flags per student.

```
CREATE VIEW Student_Eligibility_Summary AS
SELECT
    CU.Course_Code,
    CU.Course_Name,
    ST.Reg_No,
    P.F_Name,
    P.L_Name,
    ER.attendance_percentage,
    CASE WHEN ER.attendance_percentage >= 80.00 THEN 'Eligible' ELSE 'Not Eligible' END
AS Attendance_Status,
    ER.ca_mark,
    ER.ca_eligibility,
    ER.final_exam_eligibility
FROM ELIGIBILITY_RECORD ER
JOIN STUDENT ST ON ER.Student_ID = ST.Student_ID
JOIN PERSON P ON ST.Person_ID = P.Person_ID
JOIN COURSE_OFFERING CO ON ER.Offering_ID = CO.Offering_ID
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID
ORDER BY CU.Course_Code, ST.Reg_No;
```

11. Student_Grades_Summary final mark, letter grade, grade point, result code per student.

```
CREATE VIEW Student_Grades_Summary AS
SELECT
    ST.Reg_No,
    CU.Course_Code,
    CU.Course_Name,
    CU.Credits,
    FR.final_mark,
    FR.letter_grade,
    FR.grade_point,
    FR.result_code
FROM FINAL_RESULT FR
JOIN STUDENT ST ON FR.Student_ID = ST.Student_ID
JOIN COURSE_OFFERING CO ON FR.Offering_ID = CO.Offering_ID
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID
ORDER BY ST.Reg_No, CU.Course_Code;
```

12. Batch_GPA_Summary SGPA and CGPA for all students, ordered by CGPA descending.

```
CREATE VIEW Batch_GPA_Summary AS
SELECT
    ST.Reg_No,
    P.F_Name,
    P.L_Name,
    SEM.Academic_Year,
    SEM.Semester_No,
    GR.sgpa,
    GR.cgpa
FROM GPA_RECORD GR
JOIN STUDENT ST ON GR.student_id = ST.Student_ID
JOIN PERSON P ON ST.Person_ID = P.Person_ID
JOIN SEMESTER SEM ON GR.semester_id = SEM.Semester_ID
ORDER BY GR.cgpa DESC;
```

13. Detailed_Academic_Transcript semester, course, grade, credits, SGPA, CGPA — full student transcript.

```
CREATE VIEW Detailed_Academic_Transcript AS
SELECT
    ST.Reg_No,
    SEM.Academic_Year,
    SEM.Semester_No,
    CU.Course_Code,
    CU.Course_Name,
    CU.Credits,
    FR.letter_grade,
    FR.grade_point,
```

```

    GR.sgpa,
    GR.cgpa
FROM FINAL_RESULT FR
JOIN STUDENT ST ON FR.Student_ID = ST.Student_ID
JOIN COURSE_OFFERING CO ON FR.Offering_ID = CO.Offering_ID
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID
JOIN SEMESTER SEM ON CO.Semester_ID = SEM.Semester_ID
JOIN GPA_RECORD GR ON ST.Student_ID = GR.student_id AND SEM.Semester_ID =
GR.semester_id
ORDER BY ST.Reg_No, CU.Course_Code;

```

OTHER VIEWS created by Nirmal (TG2097)

14. Student_Profile_View combined student personal data from PERSON and STUDENT.

```

CREATE OR REPLACE VIEW Student_Profile_View AS
SELECT
    S.Reg_no AS Reg_No,
    CONCAT(P.F_Name, ' ', P.L_Name) AS Full_Name,
    P.F_Name,
    P.L_Name,
    P.NIC,
    P.Gender,
    P.DOB,
    P.Phone_No,
    P.Address,
    S.Batch,
    S.Intake_Year AS Academic_Year,
    S.Status AS Student_Status
FROM STUDENT S
JOIN PERSON P ON S.Person_ID = P.Person_ID;

```

15. Low_Attendance_View students with attendance below 80%, worst first. Used by lecturers and admin.

```

CREATE OR REPLACE VIEW Low_Attendance_View AS
SELECT
    S.Reg_no AS Reg_No,
    CONCAT(P.F_Name, ' ', P.L_Name) AS Student_Name,
    S.Batch,
    CU.Course_Code,
    CU.Course_Name,
    ER.attendance_percentage,
    ER.final_exam_eligibility
FROM ELIGIBILITY_RECORD ER
JOIN STUDENT S ON ER.Student_ID = S.Student_ID
JOIN PERSON P ON S.Person_ID = P.Person_ID
JOIN COURSE_OFFERING CO ON ER.Offering_ID = CO.Offering_ID
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID

```



```
WHERE ER.attendance_percentage < 80.00  
ORDER BY ER.attendance_percentage ASC;
```

16. Medical_Status_View medical submissions with course, session type, and approval status.

```
CREATE VIEW Medical_Status_View AS  
SELECT  
    M.Medical_ID,  
    S.Reg_No,  
    CONCAT(P.F_Name, ' ', P.L_Name) AS Student_Name,  
    M.submitted_date,  
    M.reason,  
    CU.Course_Code,  
    CC.component_type AS Session_Type,  
    SE.week_no,  
    SE.session_date AS Absent_Date,  
    M.approval_status  
FROM MEDICAL_RECORD M  
JOIN STUDENT S ON M.Student_ID = S.Student_ID  
JOIN PERSON P ON S.Person_ID = P.Person_ID  
JOIN SESSION SE ON M.Session_ID = SE.Session_ID  
JOIN COURSE_COMPONENT CC ON SE.Component_ID = CC.Component_ID  
JOIN COURSE_OFFERING CO ON CC.offering_id = CO.Offering_ID  
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID;
```

17. Session_Details_View session schedule with course name, component type, and department.

```
CREATE VIEW Session_Details_View AS  
SELECT  
    SE.Session_ID,  
    CC.component_type AS Session_Type,  
    SE.week_no,  
    SE.session_date,  
    SE.start_time,  
    SE.end_time,  
    CU.Course_Code,  
    CU.Course_Name,  
    CU.Credits,  
    CU.Course_Type,  
    D.Department_Name  
FROM SESSION SE  
JOIN COURSE_COMPONENT CC ON SE.Component_ID = CC.Component_ID  
JOIN COURSE_OFFERING CO ON CC.offering_id = CO.Offering_ID  
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID  
JOIN DEPARTMENT D ON CU.Department_ID = D.Department_ID;
```

18. Top_Performers_View students who scored 80 or above in any subject, ranked by final mark.

```
CREATE VIEW Top_Performers_View AS
SELECT
    S.Reg_No,
    CONCAT(P.F_Name, ' ', P.L_Name) AS Student_Name,
    S.Batch_Year AS Batch,
    CU.Course_Code,
    CU.Course_Name,
    FR.final_mark,
    FR.letter_grade AS Grade
FROM FINAL_RESULT FR
JOIN STUDENT S ON FR.Student_ID = S.Student_ID
JOIN PERSON P ON S.Person_ID = P.Person_ID
JOIN COURSE_OFFERING CO ON FR.Offering_ID = CO.Offering_ID
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID
WHERE FR.final_mark >= 80.00
ORDER BY FR.final_mark DESC;
```

6. Architecture of the Solution

The solution uses a two-layer architecture: a MySQL 8.x database server hosting the FOT_Academic_Management_DB, and an application/client layer that connects via the standard MySQL protocol.

6.1 Database Layer

- Engine: InnoDB (full ACID compliance, foreign key support, row-level locking)
- 24 relational tables organised in a 7-step dependency order
- 3 stored procedures for automated business logic (eligibility, results, GPA)
- 15+ database views providing safe, filtered read access
- Role-based MySQL user accounts with GRANT-level access control

6.2 Data Flow End to End

Step	Actor	Action
1	Admin	Creates PERSON, STAFF, STUDENT, USER_ACCOUNT records
2	Admin	Defines DEPARTMENT, SEMESTER, COURSE_UNIT records
3	Admin / Lecturer	Creates COURSE_OFFERING and assigns Lecturer; defines COURSE_COMPONENTs
4	Admin	Enrolls students: inserts ENROLLMENT rows (Student_ID, Offering_ID, attempt_no)
5	Lecturer	Defines ASSESSMENT_SCHEME (weight %, max_marks) per offering
6	Tech Officer	Creates SESSION rows; marks ATTENDANCE_RECORD per session per student
7	Student	Submits MEDICAL_RECORD linked to specific absent sessions
8	Lecturer	Enters STUDENT_MARK for each student per scheme
9	System (Procedure)	CALL Generate_Dynamic_Eligibility() — computes attendance_percentage and ca_mark
10	System (Procedure)	CALL Generate_Dynamic_Final_Result() — computes final_mark, assigns grade from GRADE_SCALE
11	System (Procedure)	CALL Generate_Dynamic_GPA() — computes credit-weighted SGPA and CGPA
12	Authorised Users	Query views (e.g. Detailed_Academic_Transcript, Final_Marks_Summary, Low_Attendance_View)

7. Tools and Technologies Used

Tool / Technology	Purpose
MySQL 8.x	Primary relational DBMS — tables, views, stored procedures, user accounts
MySQL Workbench	IDE for schema design, query execution, and user management
InnoDB Storage Engine	ACID-compliant transactions and foreign key enforcement
SQL (DDL / DML / DCL)	Table creation, data manipulation, and access control
Stored Procedures (PL/SQL)	Automated eligibility, final result, and GPA computation
Views	Read-only summarised access to derived data for different user roles
Git / GitHub	Version control with branches per module (Data_INSERT, Procedure, View, Dev)
draw.io	ER/EER and relational mapping diagram creation
Microsoft Word	Project documentation and reporting

8. Security Measures

8.1 Role-Based Access Control (RBAC)

All access to the database is controlled through MySQL user accounts. The principle of least privilege is strictly applied — no application or user connects as root.

User Role	Permissions Granted	Reason
Admin (admin_nalin)	ALL PRIVILEGES ON *.* WITH GRANT OPTION	Full system control including creating/revoking other user accounts
Dean (dean_saman)	ALL PRIVILEGES ON FOT_Academic_Management_DB.*	Needs full read/write for oversight, reporting, and approval workflows
Lecturer (lec_amali)	ALL PRIVILEGES ON FOT_Academic_Management_DB.*	Needs to enter marks, manage offerings, and view student data
Technical Officer (to_kamal)	SELECT, INSERT, UPDATE on ATTENDANCE_RECORD, SESSION; SELECT on attendance views only	Scope limited strictly to attendance management — no access to marks or results
Students (tg2095 etc.)	SELECT on Student_Profile_View, Student_Eligibility_Summary, Final_Marks_Summary, Student_Grades_Summary, Detailed_Academic_Transcript only	Read-only, through views only — cannot read raw tables or other students' data

8.2 Password Hashing

Passwords stored in the USER_ACCOUNT table should be hashed at the application layer (e.g. using bcrypt) before insertion. The VARCHAR(255) column accommodates standard hash lengths. Storing plain-text passwords in the database was used only during development and testing.

8.3 Referential Integrity

All 24 tables use InnoDB foreign keys. Orphaned records are prevented by the DBMS — for example, an ATTENDANCE_RECORD cannot reference a SESSION or STUDENT that does not exist, and deleting a DEPARTMENT would fail if students or staff reference it.

8.4 ENUM and NOT NULL Constraints

The Gender column in PERSON uses ENUM('Male','Female','Other') to restrict invalid values. Critical fields (F_Name, L_Name, NIC, hire_date, DOB, etc.) are NOT NULL. UNIQUE constraints on NIC, Reg_no, staff_no, and UserName prevent duplicate records.

8.5 View-Based Data Abstraction

Students never touch base tables. They only query predefined views that already filter data to the relevant scope. This prevents horizontal privilege escalation (one student reading another student's records).

9. Database Accounts / Users

The following MySQL user accounts were created and configured with role-appropriate permissions. This is the actual script from the MYSQL USERS file in the project repository.

Reasons for Creating These Accounts

Account	Reason
admin_nalin	System-wide administrator. Manages all user accounts, database structure, and has GRANT OPTION to delegate permissions.
dean_saman	Faculty Dean requires complete read/write access to all academic data for oversight, reporting, and final approvals.
lec_amali (+ other lecturers)	Lecturers need to enter marks for their offerings, manage assessment schemes, and view student results.
to_kamal (Technical Officer)	Manages lab sessions and records attendance. Restricted strictly to attendance tables and views — no access to marks or results.
tg2095–tg2113 (Students)	Can only view their own profile, eligibility, marks, grades, and transcript through predefined read-only views. No write access whatsoever.

10. Code Snippets Stored Procedures

Three stored procedures automate the core computational business logic. They must be called in order: Eligibility → Final Result → GPA.

10.1 Generate_Dynamic_Eligibility (TG2097 Nirmal)

This procedure clears and repopulates ELIGIBILITY_RECORD. For each enrollment, it computes attendance_percentage counting Present and Medical statuses ca_mark the weighted sum of CA-type assessments (Quiz, Project, Mid Theory, Mid Practical); and (c) the eligibility flags based on the 80% attendance threshold.

```
DROP PROCEDURE IF EXISTS Generate_Dynamic_Eligibility;

DELIMITER $$

CREATE PROCEDURE Generate_Dynamic_Eligibility()
BEGIN

    DELETE FROM ELIGIBILITY_RECORD;
    INSERT INTO ELIGIBILITY_RECORD (Eligibility_ID, Student_ID, Offering_ID, attendance_percentage,
ca_mark, ca_eligibility, final_exam_eligibility)
    SELECT

        CONCAT('ELI ', E.Offering_ID, ' ', E.Student_ID),
        E.Student_ID,
        E.Offering_ID,

        COALESCE((
            SELECT (SUM(CASE WHEN A.attendance_status IN ('Present', 'Medical') THEN 1 ELSE 0 END) /
COUNT(*) * 100
            FROM ATTENDANCE_RECORD A
            JOIN SESSION S ON A.session_id = S.Session_ID
            JOIN COURSE_COMPONENT C ON S.Component_ID = C.Component_ID
            WHERE A.student_id = E.Student_ID AND C.offering_id = E.Offering_ID
        ), 0) AS attendance_percentage,

        COALESCE((
            SELECT SUM(SM.raw_mark * (ASCH.weight_percentage / 100))
            FROM STUDENT_MARK SM
            JOIN ASSESSMENT_SCHEME ASCH ON SM.Scheme_ID = ASCH.Scheme_ID
            WHERE SM.Student_ID = E.Student_ID
            AND ASCH.Offering_ID = E.Offering_ID

            AND ASCH.Assessment_Type_ID IN ('AT_QZ', 'AT_PR', 'AT_MT', 'AT_MP')
        ), 0) AS ca_mark,
```

```

FALSE,
FALSE
FROM ENROLLMENT E;

UPDATE ELIGIBILITY_RECORD
SET ca_eligibility = IF(attendance_percentage >= 80.00, TRUE, FALSE),
    final_exam_eligibility = IF(attendance_percentage >= 80.00, TRUE, FALSE);

END$$
DELIMITER ;
CALL Generate_Dynamic_Eligibility();

```

10.2 Generate_Dynamic_Final_Result (TG2096 Himansana)

This procedure clears and repopulates FINAL_RESULT. Students who are not eligible receive a final_mark of 0.00. Eligible students receive the full weighted sum of all assessment marks. Grades are then assigned by joining with GRADE_SCALE using FLOOR() to match integer ranges.

```

DELIMITER $$

CREATE PROCEDURE Generate_Dynamic_Final_Result()
BEGIN

    DELETE FROM FINAL_RESULT;

    INSERT INTO FINAL_RESULT (Result_ID, Student_ID, Offering_ID, final_mark, letter_grade, grade_point,
result code, released date)
    SELECT

        CONCAT('RES_', E.Offering_ID, '_', E.Student_ID) AS Result_ID,
        E.Student_ID,
        E.Offering_ID,

        CASE
            WHEN ELI.final_exam_eligibility = FALSE THEN 0.00
            ELSE COALESCE((
                SELECT SUM(SM.raw_mark * (ASCH.weight_percentage / 100))
                FROM STUDENT_MARK SM
                JOIN ASSESSMENT_SCHEME ASCH ON SM.Scheme_ID = ASCH.Scheme_ID
                WHERE SM.Student_ID = E.Student_ID AND ASCH.Offering_ID = E.Offering_ID
            ), 0)
        END AS final_mark,

```

```

'TBD' AS letter_grade,
0.00 AS grade_point,
'TBD' AS result_code,
CURDATE() AS released_date

FROM ENROLLMENT E
JOIN ELIGIBILITY_RECORD ELI ON E.Student_ID = ELI.Student_ID AND E.Offering_ID = ELI.Offering_ID;

UPDATE FINAL_RESULT FR
JOIN GRADE_SCALE GS ON FLOOR(FR.final_mark) BETWEEN GS.min_mark AND GS.max_mark
SET
    FR.letter_grade = GS.letter_grade,
    FR.grade_point = GS.grade_point,

    FR.result_code = IF(FR.final_mark >= 35.00, 'PASS', 'FAIL');

END$$

DELIMITER ;

CALL Generate_Dynamic_Final_Result();

```

10.3 Generate_Dynamic_GPA (TG2095 Hasaranga)

This procedure clears and repopulates GPA_RECORD. For each student per semester, it computes the credit-weighted SGPA using the grade points from FINAL_RESULT and the credit values from COURSE_UNIT. As this is a single-semester system, CGPA equals SGPA.

```

DROP PROCEDURE IF EXISTS Generate_Dynamic_GPA;

DELIMITER $$

CREATE PROCEDURE Generate_Dynamic_GPA()
BEGIN

    DELETE FROM GPA_RECORD;

    INSERT INTO GPA_RECORD (gpa_id, student_id, semester_id, sgpa, cgpa)
    SELECT
        CONCAT('GPA_', CO.Semester_ID, '_', FR.Student_ID) AS gpa_id,
        FR.Student_ID AS student_id,
        CO.Semester_ID AS semester_id,

```



```
CAST((SUM(CU.Credits * FR.grade_point) / SUM(CU.Credits)) AS DECIMAL(4,2)) AS sgpa,  
CAST((SUM(CU.Credits * FR.grade_point) / SUM(CU.Credits)) AS DECIMAL(4,2)) AS cgpa  
  
FROM FINAL RESULT FR  
JOIN COURSE OFFERING CO ON FR.Offering_ID = CO.Offering_ID  
JOIN COURSE_UNIT CU ON CO.Course_ID = CU.Course_ID  
GROUP BY FR.Student_ID, CO.Semester_ID;  
  
END$$  
  
DELIMITER ;  
  
CALL Generate_Dynamic_Eligibility();  
CALL Generate_Dynamic_Final_Result();  
CALL Generate_Dynamic_GPA();  
  
SELECT * FROM GPA_RECORD;
```

10.4 Additional Stored Procedures Report Generation & Data Retrieval

The following seven stored procedures provide parameterised report generation for the most common queries required by Admins, Lecturers, Technical Officers, and Students. Each procedure queries one of the views defined in Section 5.3, ensuring that the business logic remains centralised and consistent.

10.4 SP_Get_Students_By_Eligibility Eligible / Not Eligible Students for a Course (TG2095 -Hasaranga)

Returns all students in a given course who are either eligible or not eligible for the final exam. Accepts the course code and a boolean flag (TRUE = Eligible, FALSE = Not Eligible). Queries the Student_Eligibility_Summary view.

```
CREATE PROCEDURE SP_Get_Students_By_Eligibility(
    IN p_Course_Code VARCHAR(10),
    IN p_Is_Eligible BOOLEAN
)
BEGIN
    SELECT Reg_No, F_Name, L_Name, Course_Code, attendance_percentage, ca_mark,
           CASE WHEN final_exam_eligibility = 1 THEN 'Eligible' ELSE 'Not Eligible' END AS Eligibility
    FROM Student_Eligibility_Summary
    WHERE Course_Code = p_Course_Code AND final_exam_eligibility = p_Is_Eligible;
END$$
```

10.5 SP_Get_Student_Final_Marks Final Marks and Grades for a Student (All Subjects) (TG2095 - Hasaranga)

Returns the final mark, letter grade, and result code for all subjects of a specific student. Queries the Final_Marks_Summary view.

```
CREATE PROCEDURE SP_Get_Student_Final_Marks(
    IN p_Reg_No VARCHAR(20)
)
BEGIN
    SELECT Reg_No, Course_Code, Course_Name, final_mark, letter_grade, result_code
    FROM Final_Marks_Summary
    WHERE Reg_No = p_Reg_No;
END$$
```

10.6 SP_Get_One_Student_All_Attendance Attendance of a Student Across All Subjects (TG2096 – Himansana)

Returns the attendance percentage and eligibility status for a specific student across all enrolled subjects. Queries the Student_Eligibility_Summary view.

```
CREATE PROCEDURE SP_Get_One_Student_All_Attendance(
    IN p_Reg_No VARCHAR(20)
)
BEGIN
    SELECT Reg_No, Course_Code, Course_Name, attendance_percentage, Attendance_Status
    FROM Student_Eligibility_Summary
    WHERE Reg_No = p_Reg_No;
END$$
```

10.7 SP_Get_Student_CA_Course CA Mark of a Student for a Specific Course (TG2096 – Himansana)

Returns the CA mark and CA eligibility flag for one student in one specific course. Queries the Student_CA_Marks_Summary view.

```
CREATE PROCEDURE SP_Get_Student_CA_Course(  
    IN p_Reg_No VARCHAR(20),  
    IN p_Course_Code VARCHAR(10)  
)  
BEGIN  
    SELECT Reg_No, Course_Code, Course_Name, ca_mark, ca_eligibility  
    FROM Student_CA_Marks_Summary  
    WHERE Reg_No = p_Reg_No AND Course_Code = p_Course_Code;  
END$$
```

10.8 SP_Get_Batch_CA_Course CA Marks of the Entire Batch for a Specific Course (TG2097 – Nirmal)

Returns the CA mark and CA eligibility for every student in a given course. Queries the Batch_CA_Marks_Summary view. Useful for lecturers to review the batch before releasing results.

```
CREATE PROCEDURE SP_Get_Batch_CA_Course(  
    IN p_Course_Code VARCHAR(10)  
)  
BEGIN  
    SELECT Reg_No, F_Name, L_Name, Course_Code, ca_mark, ca_eligibility  
    FROM Batch_CA_Marks_Summary  
    WHERE Course_Code = p_Course_Code;  
END$$
```

10.9 SP_Get_Batch_Attendance_Course Attendance of the Entire Batch for a Specific Course Student (TG2112 - Sarathchandra GSM)

Returns the attendance percentage and eligibility status for all students enrolled in a given course. Queries the Batch_Attendance_Summary view. Used by Technical Officers and Lecturers for attendance reporting.

```
CREATE PROCEDURE SP_Get_Batch_Attendance_Course(  
    IN p_Course_Code VARCHAR(10)  
)  
BEGIN  
    SELECT Reg_No, Course_Code, Attendance_Percentage, Eligibility_Status  
    FROM Batch_Attendance_Summary  
    WHERE Course_Code = p_Course_Code;  
END$$
```

10.10 SP_Get_Student_Medicals All Medical Records Submitted by a Student (TG2112 - Sarathchandra GSM)

Returns all medical submissions for a specific student, including the absent session date, course code, reason, and approval status. Queries the Medical_Status_View.

```
CREATE PROCEDURE SP_Get_Student_Medicals(
  IN p_Reg_No VARCHAR(20)
)
BEGIN
  SELECT Medical_ID, Reg_No, Student_Name, submitted_date, reason, Course_Code, Absent_Date,
approval_status
  FROM Medical_Status_View
  WHERE Reg_No = p_Reg_No;
END$$
```

11. Problems Faced During Development

#	Problem Encountered
1	Foreign key dependency ordering: MySQL raises errors when creating a table that references another table not yet created. With 24 tables and complex cross-references, determining the correct creation order initially resulted in multiple FK violation errors.
2	The M:N relationship between STUDENT and COURSE_OFFERING was initially modelled without an associative entity, causing schema issues. It was later resolved by introducing the ENROLLMENT table with its own PK and attempt tracking.
3	CA mark computation in the stored procedure required filtering only CA-type assessments (AT_QZ, AT_PR, AT_MT, AT_MP). An initial version summed all assessments including finals, producing incorrect CA marks.
4	Attendance percentage initially only counted 'Present' status. The faculty policy of counting approved medical absences as attended was missed, leading to lower-than-expected attendance percentages and incorrect eligibility flags.
5	Git merge conflicts arose because four team members worked on separate branches (Data_INSERT, Procedure, View, Dev) and SQL files in the same folder were modified by multiple members simultaneously.
6	The GRADE_SCALE BETWEEN comparison on a DECIMAL final_mark column against INT range columns produced edge-case rounding errors (e.g. 84.99 not matching G_A's range of 75–84). The fix was wrapping final_mark with FLOOR() before the comparison.
7	MySQL's INT(11) deprecation warning appeared for Office_Phone in DEPARTMENT during testing on MySQL 8.0.17+. This was documented and flagged for migration to BIGINT in the production schema.
8	The DEAN table initially included an Office_Extension column that was present in the ER diagram but absent from the relational mapping. The inconsistency was resolved by aligning both diagrams in the final version.

12. Solutions to the Identified Problems

#	Solution Applied
1	A dependency-ordered table creation document (FULL_TABLE_CREATION) was produced listing all 7 steps from base tables to derived tables. Each team member ran the script in this order only.
2	The ENROLLMENT associative entity was added with its own Enrollment_ID PK, FKs to STUDENT and COURSE_OFFERING, and an attempt_no column to support repeat enrollments. 225 enrollment records (25 students × 9 offerings) were inserted.
3	Assessment Type IDs for CA were standardised as AT_QZ, AT_PR, AT_MT, AT_MP. The stored procedure was updated to filter ASCH.Assessment_Type_ID IN ('AT_QZ','AT_PR','AT_MT','AT_MP'), excluding finals.
4	The attendance query was updated to count WHERE attendance_status IN ('Present','Medical'), matching the faculty's official policy.
5	A branch-per-feature strategy was enforced. TG2095 (Hasaranga) was designated team lead and handled all merges to main after a team review. Merge conflicts were resolved manually before committing.
6	FLOOR(FR.final_mark) was applied before the BETWEEN comparison, ensuring that marks like 84.99 map correctly to the A grade band (75–84) without floating-point boundary issues.
7	Documented as a known issue for production migration. The column will be changed to BIGINT in the cloud-deployed version.
8	The FULL_TABLE_CREATION script was used as the single source of truth for all table definitions. Any discrepancies between individual member files were resolved against this master script.

13. Backend Hosting

13.1 Planned Hosting Platform: Amazon RDS for MySQL

The backend database is planned to be hosted on Amazon Web Services (AWS) using Amazon RDS for MySQL. During development and evaluation, the RDS Free Tier (db.t3.micro, 20 GB SSD, Single-AZ) will be used. For production, it can be scaled to Multi-AZ with automated failover.

13.2 Reasons for Selecting AWS RDS

- **Managed service:** AWS handles OS patching, database engine upgrades, automated daily snapshots, and point-in-time recovery automatically.
- **MySQL compatibility:** All existing DDL scripts, stored procedures, and views run on RDS for MySQL without modification.
- **Scalability:** Can scale instance size vertically or add read replicas horizontally as student population grows.
- **Security:** Supports VPC isolation, security groups (port 3306 whitelisting), SSL/TLS connections, and IAM database authentication.
- **Multi-AZ:** Automatic failover to a standby replica in a different availability zone ensures high availability for exam season.
- **Free Tier:** Suitable for academic project submission and testing at zero cost during the evaluation period.

13.3 Changes Required for Cloud Deployment

Change Required	Detail
User host binding	Change all 'user'@'localhost' to 'user'@'%' or restrict to the application server's specific IP/VPC CIDR range.
Password strength	Enable MySQL validate_password plugin via the RDS parameter group; enforce strong passwords for all user accounts.
SSL/TLS connections	Configure the MySQL client to require SSL. Download and reference the AWS RDS CA bundle (rds-ca-2019-root.pem) in connection strings.
Connection strings	Update all application connection strings to use the RDS endpoint hostname (e.g. fot-db.xxxx.rds.amazonaws.com) instead of 'localhost'.
RDS Parameter Group	Tune max_connections, innodb_buffer_pool_size, and slow_query_log based on expected concurrent user load.
Automated backups	Configure automated daily snapshots with 7-day retention and enable point-in-time recovery.
VPC & Security Groups	Restrict inbound port 3306 to the application server's security group only — never expose the RDS endpoint publicly.
INT(11) column migration	Change Office_Phone in DEPARTMENT from INT(11) to BIGINT to eliminate MySQL 8.x deprecation warnings in the production schema.

14. Individual Contribution to Backend Development

14.1 Contribution of Hasaranga K.H.M (Team Leader)



Index No: TG/2024/2095

Main Role: Project Coordination, Git Infrastructure Management, and Core Backend Architecture.

Overall Impact:

As the Team Leader, I established the project's technical framework and managed the collaborative development workflow using GitHub. I implemented a robust branching strategy, including dev for core development, Data_INSERT for population, and specialized branches for View and Procedure logic to ensure code integrity and conflict-free merging. My contribution focused on the system's structural normalization and the complex logic required for GPA and enrollment tracking.

Key Tasks Performed:

- **Git Repository Management:** Configured and maintained the central repository, managing merges and resolving version control conflicts across four feature branches.
- **Relational Schema Design:** Directed the translation of the ER/EER diagrams into a normalized 3NF relational database structure.
- **Core Computational Logic:** Developed the logic for GPA calculations and student enrollment tracking to ensure academic data accuracy.

Database Objects Created:

• Tables:

- **ASSESSMENT_TYPE:** Defines various assessment categories like Quiz, Project, and Final Exams.
- **GRADE_SCALE:** Stores the official mark-to-grade mapping and grade point values.
- **COURSE_OFFERING:** Manages the specific instances of courses offered to batches by lecturers.
- **COURSE_COMPONENT:** Breaks down offerings into Theory and Practical components.
- **ENROLLMENT:** An associative entity tracking student attempts and status for course offerings.
- **STUDENT_MARK:** Records individual marks for each student per assessment scheme.
- **ELIGIBILITY_RECORD:** Stores computed attendance percentages and CA marks for exam qualification.
- **FINAL_RESULT:** Contains the final weighted marks and released grades for each course.
- **GPA_RECORD:** Stores semester-wise SGPA and cumulative CGPA per student.
- **ASSESSMENT_SCHEME:** Defines the weightage and mandatory status for various assessments.

- **Views:**

- **Batch_Attendance_Summary:** Provides an overview of attendance and eligibility for an entire batch.
- **Component_Wise_Attendance:** Displays attendance percentages split between Theory and Practical sessions.
- **Individual_Attendance_Summary:** Summarizes attendance percentage for a specific student across subjects.
- **Individual_Subject_Attendance_Detail:** Shows session-by-session attendance history for individual students.

- **Stored Procedures:**

- **Generate_Dynamic_GPA:** Automatically computes credit-weighted SGPA and CGPA for all students.
- **SP_Get_Students_By_Eligibility:** Retrieves lists of students eligible or ineligible for exams in a specific course.
- **SP_Get_Student_Final_Marks:** Generates a report of final marks and grades for a specific student registration number.

GitHub Link: [<https://github.com/Hasaranga-kariyawasam>]

14.2 Contribution of Himansana N.V.S



Index No: TG/2024/2096

Main Role: Result Generation Logic and Role-Based Access Control (RBAC).

Overall Impact:

Himansana was responsible for the critical automation of final results and the security layer of the database. By developing advanced result-calculation procedures and configuring role-based user accounts, he ensured the system is both functional and secure from unauthorized access.

Key Tasks Performed:

- **Security & Access Control:** Authored the MYSQL USERS script to implement the principle of least privilege for different user roles.
- **Automated Result Computation:** Developed stored procedures to process raw marks into final grades using floor-rounded comparisons.
- **Data Initialization:** Managed the population of primary staff, department, and semester entities.

Database Objects Created:

- **Views:**
 - **Student_Eligibility_Summary:** Shows attendance status, CA marks, and eligibility flags per student.
 - **Student_Grades_Summary:** Provides a summary of final marks, grades, and result codes for each student.
 - **Batch_GPA_Summary:** Ranks the entire batch based on CGPA for academic performance review.
 - **Detailed_Academic_Transcript:** Generates a full transcript including grades, credits, and GPA records.
- **Stored Procedures:**
 - **Generate_Dynamic_Final_Result:** Computes total weighted marks and assigns letter grades based on eligibility.
 - **SP_Get_One_Student_All_Attendance:** Reports a specific student's attendance percentage and eligibility status across all subjects.
 - **SP_Get_Student_CA_Course:** Retrieves the CA mark and eligibility status for a student in a particular course.

GitHub Link: [<https://github.com/himansana2004>]

14.3 Contribution of Nirmal I.D.N.S



Index No: TG/2024/2097

Main Role: User Hierarchy Management and Eligibility Computation.

Overall Impact:

Nirmal focused on the fundamental identity management of the system, designing the generalization hierarchy for diverse personnel. He implemented the essential logic that determines exam eligibility by integrating medical and session attendance data, ensuring institutional policies are met.

Key Tasks Performed:

- **User Specialization Modeling:** Designed and created the tables for specialized staff roles including Admin, Dean, and Technical Officer.
- **Eligibility Logic Implementation:** Developed procedures to calculate qualification status based on attendance thresholds.
- **Comprehensive Mark Entry:** Managed the bulk insertion of student mark data to test reporting views.

Database Objects Created:

- **Tables:**
 - PERSON, STAFF, STUDENT, USER_ACCOUNT: Foundation tables for managing personal and login data.
 - ADMIN, DEAN, LECTURER, TECHNICAL_OFFICER: Specialized tables for different university personnel roles.
- **Views:**
 - **Student_Profile_View:** Combines personal details with academic registration information.
 - **Low_Attendance_View:** Identifies students with attendance below 80% for academic intervention.
 - **Medical_Status_View:** Tracks submitted medical records and their respective approval statuses.
 - **Session_Details_View:** Provides a comprehensive schedule including course details and locations.
 - **Top_Performers_View:** Lists high-achieving students who scored 80 or above in any subject.
- **Stord Procedures:**
 - **Generate_Dynamic_Eligibility:** Automatically clears and computes student CA marks and final exam eligibility.
 - **SP_Get_Batch_CA_Course:** Provides a report of CA marks and eligibility for all students in a given batch.

GitHub Link: [<https://github.com/nirmalsasindu40>]

14.4 Contribution of Sarathchandra G.M.S



Index No: TG/2024/2112

Main Role: Academic Core Infrastructure and Marks Analysis.

Overall Impact:

Sarathchandra was responsible for building the core academic components of the database, from department definitions to session records. He developed advanced marks views that allow for multi-dimensional analysis of student performance, facilitating efficient reporting for lecturers and admins.

Key Tasks Performed:

- **Infrastructure Setup:** Created the baseline tables for university departments, semesters, and individual course units.
- **Attendance Tracking Logic:** Implemented the storage for sessions and attendance records to support eligibility flows.
- **Data Integration:** Managed the population of 225 enrollment records and session scheduling data.

Database Objects Created:

- **Tables:**
 - DEPARTMENT, SEMESTER, COURSE_UNIT: Essential academic organizational tables.
 - SESSION, ATTENDANCE_RECORD, MEDICAL_RECORD: Core tables for tracking daily academic activity and absences.
- **Views:**
 - **Individual_CA_Marks_Detail:** Provides a detailed weighted breakdown of all CA components per subject.
 - **Student_CA_Marks_Summary:** Displays summarized CA results and eligibility for students.
 - **Batch_CA_Marks_Summary:** Summarizes CA marks for an entire batch for lecturer review.
 - **Comprehensive_Marks_Breakdown:** Pivots all assessment types into columns for a side-by-side comparison.
 - **Final_Marks_Summary:** A complete academic record view combining attendance, CA, and final results.
- **Stored Procedures:**
 - **SP_Get_Batch_Attendance_Course:** Retrieves the attendance percentage and eligibility for a whole batch in a specific course.
 - **SP_Get_Student_Medicals:** Lists all medical submissions for a student including reasons and approval status.

GitHub Link: [<https://github.com/samindi2004>]

15. References

- Elmasri, R. & Navathe, S.B. (2015). Fundamentals of Database Systems (7th ed.). Pearson.
- Silberschatz, A., Korth, H.F. & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.
- MySQL 8.0 Reference Manual. Oracle Corporation. <https://dev.mysql.com/doc/refman/8.0/en/>
- MySQL Workbench Manual. Oracle Corporation. <https://dev.mysql.com/doc/workbench/en/>
- AWS Documentation — Amazon RDS for MySQL.
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_MySQL.html
- DBMS Practicum Module Notes — Faculty of Technology, University of Sri Jayewardenepura (2024).

GitHub Repo Link: [https://github.com/Hasaranga-kariyawasam/DBMS_MINI_PROJECT]